

Python Data Structures – Hands-on Practice

Learn → Explore → Practice

Learner reference created from the supplied Jupyter notebook.

Python Data Structures – Hands-on Practice

Learn → Explore → Practice

This notebook combines interactive Python practice with instructional content from the accompanying Rise course.

Theory

The Rise lesson introduces the four core Python data structures:

- Lists
- Tuples
- Sets
- Dictionaries

Learn by Exploring, Testing, and Solving

In this hands-on practice, we will explore Python's core data structures through small examples and interactive exercises.

Topics covered:

- Lists
- Tuples
- Sets
- Dictionaries

The goal is not just to learn the syntax, but to understand **when and why to use each data structure**.

List

```
# A list stores multiple values in a single variable  
fruits = ["apple", "banana", "orange", "mango"]  
print(fruits)
```

Output

```
['apple', 'banana', 'orange', 'mango']
```

1. Exploring Lists

A Python list can store multiple values in a single variable.

Lists are:

- **Ordered** – items keep their position.
- **Mutable** – items can be changed after the list is created.
- **Indexed** – each item has a position starting from 0.

Try it yourself

Run the next code cell and observe what happens when you access different items in the list.

```
print(fruits[0])  
print(fruits[2])
```

Output

```
apple  
orange
```

Challenge 1 – Change an Item

Lists are **mutable**, which means we can change an existing item.

The fruit banana is currently at index 1.

Your task

Change banana to grapes.

Try writing the code yourself before looking at the answer.

```
# Change banana to grapes  
  
fruits[1] = _____  
  
print(fruits)
```

Check your answer

The completed code should produce:

```
['apple', 'grapes', 'orange', 'mango']
```

```
fruits[1] = "grapes"  
  
print(fruits)
```

Output

```
['apple', 'grapes', 'orange', 'mango']
```

List Manipulations

Lists can be modified after they are created. We can add new items, remove existing items, and change the order of items.

Adding an item with `append()`

The `append()` method adds a new item to the **end** of a list.

```
fruits = ["apple", "grapes", "orange", "mango"]  
fruits.append("banana")  
print(fruits)
```

Output

```
['apple', 'grapes', 'orange', 'mango', 'banana']
```

Challenge 2 – Add an item

Add "watermelon" to the end of the `fruits` list using `append()`.

Try it yourself before looking at the answer.

```
# Your code here
```

Check your answer

You should use:

```
fruits.append("watermelon")
```

After running it, "watermelon" should appear at the end of the list.

```
fruits.append("watermelon")  
print(fruits)
```

Output

```
['apple', 'grapes', 'orange', 'mango', 'banana', 'watermelon']
```

Removing an Item with `remove()`

The `remove()` method removes the **first matching item** from a list.

For example, we can remove "orange" from our `fruits` list.

```
fruits = ["apple", "grapes", "orange", "mango", "banana"]  
fruits.remove("orange")  
print(fruits)
```

Output

```
['apple', 'grapes', 'mango', 'banana']
```

Challenge 3 – Remove an Item

Remove "grapes" from the `fruits` list using `remove()`.

Try it yourself before looking at the answer.

```
# Your code here
```

Check your answer

Use:

```
fruits.remove("grapes")
```

The list should no longer contain "grapes".

```
fruits.remove("grapes")  
print(fruits)
```

Output

```
['apple', 'mango', 'banana']
```

Inserting an Item with `insert()`

The `insert()` method adds an item at a **specific position** in a list.

It takes two arguments:

```
insert(index, item)
```

For example, `insert(1, "banana")` places "banana" at index 1.

```
fruits = ["apple", "grapes", "mango", "banana"]  
fruits.insert(1, "orange")  
print(fruits)
```

Output

```
['apple', 'orange', 'grapes', 'mango', 'banana']
```

Challenge 4 – Insert an Item

Insert "watermelon" at index 2 in the `fruits` list.

Try it yourself before looking at the answer.

```
# Your code here
```

Check your answer

Use:

```
fruits.insert(2, "watermelon")
```

The resulting list should be:

```
['apple', 'orange', 'watermelon', 'grapes', 'mango', 'banana']
```

```
fruits.insert(2, "watermelon")
print(fruits)
```

Output

```
['apple', 'orange', 'watermelon', 'grapes', 'mango', 'banana']
```

Removing an Item with pop()

The pop() method removes an item from a list and **returns the removed item**.

If no index is provided, pop() removes the last item.

```
fruits = ["apple", "orange", "watermelon", "grapes", "mango"]
removed_fruit = fruits.pop()

print("Removed:", removed_fruit)
print("Remaining:", fruits)
```

Output

```
Removed: mango
Remaining: ['apple', 'orange', 'watermelon', 'grapes']
```

Challenge 5 – Use pop()

Remove the item at index 1 using pop().

Store the removed item in a variable called removed_fruit.

Then print both the removed fruit and the remaining list.

```
# Your code here
```

Check your answer

Use:

```
removed_fruit = fruits.pop(1)
```

The removed item should be "orange".

```
removed_fruit = fruits.pop(1)

print("Removed:", removed_fruit)
print("Remaining:", fruits)
```

Output

```
Removed: orange  
Remaining: ['apple', 'watermelon', 'grapes']
```

Sorting a List

The `sort()` method arranges the items in a list in ascending order.

For text, this normally means alphabetical order.

```
fruits = ["mango", "apple", "orange", "banana"]  
  
fruits.sort()  
  
print(fruits)
```

Output

```
['apple', 'banana', 'mango', 'orange']
```

Challenge 6 – Sort the List

Create a list of five numbers and use `sort()` to arrange them from smallest to largest.

For example, your list might contain:

```
[42, 7, 19, 3, 25]
```

```
# Create your list here  
  
# Sort the list here  
  
# Print the result
```

Check your answer

For example:

```
numbers = [42, 7, 19, 3, 25]
```

Then use:

```
numbers.sort()
```

The result should be:

```
[3, 7, 19, 25, 42]
```

Challenge 7 – Manage a Shopping List

You are preparing a shopping list for the week.

Start with:

```
shopping = ["milk", "bread", "eggs", "rice"]
```

Complete the following tasks:

1. Add "apples" to the end of the list. 2. Insert "butter" at index 1. 3. Remove "eggs" from the list. 4. Remove the last item using pop() and store it in removed_item. 5. Sort the remaining items alphabetically. 6. Print the final list. 7. Print the item that was removed using pop().

Your task

Write the code yourself using the list methods you have learned.

```
# Start with the shopping list
shopping = ["milk", "bread", "eggs", "rice"]

# 1. Add "apples"

# 2. Insert "butter" at index 1

# 3. Remove "eggs"

# 4. Remove the last item and store it in removed_item

# 5. Sort the list

# 6. Print the final list

# 7. Print the removed item
```

Check Your Answer

A possible solution is shown below.

Think about why each method is used for the particular task.

```
shopping = ["milk", "bread", "eggs", "rice"]
shopping.append("apples")
shopping.insert(1, "butter")
shopping.remove("eggs")
removed_item = shopping.pop()
shopping.sort()

print("Final list:", shopping)
print("Removed item:", removed_item)
```

Output

```
Final list: ['bread', 'butter', 'milk', 'rice']
Removed item: apples
```

Quick Reflection

Which method would you use when you want to:

- Add an item to the end of a list?
- Add an item at a specific position?
- Remove a specific item?
- Remove an item by its position?
- Arrange the items in order?

Think about the difference between `remove()` and `pop()`.